Studying the Effectiveness of Human-Written versus LLM-Generated Unit Test Suites

Anonymous Author(s)

Abstract

Writing unit tests manually is time-consuming, costly, and error-prone, motivating the need for automated solutions. Large Language Models (LLMs) have shown promising potential for test generation by leveraging extensive learned knowledge to produce test code more efficiently. This study presents a comparative evaluation of developer-written unit test suites and unit test suites generated automatically by three state-of-the-art LLMs for five long-lived open-source projects from the Apache Software Foundation. The evaluation combines structural and behavioral metrics, including line coverage, mutation coverage, test strength, and class-level cyclomatic complexity. The experiments were conducted in a controlled setting, using the same set of projects, standardized prompts, and a constant execution environment, enabling a direct comparison among the approaches. The results indicate that LLM-generated test suites can, in some cases, achieve line-coverage levels close to, or even above, those of manually written tests. However, manually developed test suites consistently provide better defect-detection capability, particularly when evaluated using mutation coverage and test strength. The results also show that code coverage alone is not a reliable indicator of test quality and that the effectiveness of LLM-generated tests decreases for classes with higher cyclomatic complexity. Among the evaluated models, Opus 4.5 achieved the most consistent overall performance across projects. Overall, the findings suggest that LLM-generated test suites are best used as a complementary testing strategy, extending structural coverage and exploring additional execution scenarios while developer-written tests remain more effective for reliable fault detection.

Keywords

Unit test automation; Large Language Models; Code coverage; Mutation testing.

1 Introduction

Software testing plays a fundamental role in improving software quality by identifying defects before software deployment [14]. Among the different testing levels, unit testing is particularly important because it enables faults to be detected early while improving software maintainability [9]. However, developing and maintaining unit test suites remains a labor-intensive activity and is frequently deprioritized due to time and resource constraints [15, 19]. Consequently, automated unit test generation has attracted considerable attention as a means of reducing developer effort while maintaining testing effectiveness [20].

Recent advances in Artificial Intelligence (AI), particularly Large Language Models (LLMs), have significantly improved automatic code generation, including the generation of unit tests. Models such as OpenAI Codex and GitHub Copilot can generate tests directly from source code and natural-language descriptions [24, 25]. Previous studies have shown that LLM-generated test suites can achieve structural coverage comparable to developer-written tests and benefit from contextual information provided during generation [12]. Nevertheless, existing evidence also indicates that these tests often contain weak assertions, inadequate test oracles, and maintainability issues, raising questions about their ability to effectively detect software faults [4, 23].

Although recent studies have demonstrated the potential of LLMs for automated unit test generation, most evaluations emphasize structural coverage or isolated quality aspects. Consequently, it remains unclear whether high coverage actually reflects effective fault detection and how different LLMs perform when evaluated using complementary structural and behavioral metrics under the same experimental conditions.

To address this gap, we conduct a controlled empirical study comparing developer-written unit test suites with tests generated by three state-of-the-art LLMs across five Apache Commons Java projects. Rather than relying solely on structural coverage, we perform a multidimensional evaluation combining JaCoCo coverage metrics with mutation testing using PIT, including line coverage, PIT line coverage, mutation coverage, test strength, and class-level cyclomatic complexity. This enables us to investigate the relationship between code coverage, fault-detection capability, and software complexity, providing a broader assessment of the effectiveness of LLM-generated tests.

Our results show that LLM-generated test suites can achieve competitive structural coverage, but this does not necessarily translate into effective fault detection. Developer-written test suites consistently outperform LLM-generated suites in mutation-based metrics, particularly for classes with higher cyclomatic complexity. Among the evaluated models, Claude Opus 4.5 achieved the most consistent overall performance, while the results also demonstrate that mutation-based metrics provide a more reliable assessment of test quality than structural coverage alone.

This paper makes the following contributions:

(1) We provide an empirical comparison between developer-written unit tests and tests generated by different LLMs using real Java projects from the Apache Commons ecosystem.
(2) We jointly analyze structural and behavioral test-quality metrics by combining line coverage, PIT line coverage, mutation coverage, and test strength.
(3) We investigate, at the class level, how cyclomatic complexity affects the coverage and mutation effectiveness of developer-written and LLM-generated test suites.
(4) We provide evidence that LLM-generated tests can complement developer-written suites by increasing structural coverage and exercising additional scenarios, although they still present limitations in terms of defect detection, especially for complex code.

The remainder of this paper is organized as follows. Section 2 presents the background concepts on software testing and Large

Language Models for unit test generation. Section 3 reviews the related work. Section 4 describes the empirical study, including the research questions, selected projects, evaluated language models, experimental setup, and evaluation procedure. Section 5 presents and discusses the empirical results. Section 6 discusses the threats to validity and the limitations of the study. Finally, Section 7 concludes the paper and outlines directions for future work.

2 Background

This section presents the main concepts required to understand this work. Section 2.1 discusses software testing, with emphasis on unit testing. Section 2.2 addresses the use of Large Language Models in test-code generation and the role of prompt engineering in guiding model outputs.

2.1 Software Testing

Software testing is the process of verifying that a software system behaves as expected while identifying defects before deployment [18]. Different testing levels can be applied, including unit, integration, and system testing, each targeting different levels of software functionality. Early fault detection contributes to software reliability and maintainability.

Among these levels, unit testing focuses on individual software units, typically methods or classes, executed in isolation [2]. Unit tests validate the behavior of these units through assertions over expected outcomes while isolating external dependencies, when necessary, using stubs or mocks. Because they provide fast feedback and support safer software evolution, unit testing is considered a fundamental software engineering practice.

2.2 Large Language Models for Test Generation

Large Language Models (LLMs) are AI models trained on large volumes of text and source code, enabling them to learn patterns from natural and programming languages. Models such as GPT-4, Codex, and LLaMA can interpret textual instructions and generate source code in different programming languages from a given description [22].

In software testing, LLMs can generate unit tests from different types of input, including textual specifications, expected software behavior, or the source code itself. Unlike traditional test-generation approaches, which are typically guided by structural criteria or search-based techniques, LLMs rely on knowledge learned from large code and documentation corpora. As a result, they can produce test cases that are more contextualized and closer to the style of developer-written tests, often including meaningful assertions and diverse testing scenarios [10].

The quality of the generated tests depends strongly on the instructions provided to the model, making prompt engineering an essential component of LLM-based test generation. Prompt engineering consists of designing natural-language instructions that guide the model toward the desired output by specifying, for example, the target classes, testing framework, expected assertions, and generation constraints.

Well-designed prompts reduce ambiguity, improve consistency, and produce tests that are better aligned with the intended objective. Conversely, generic or poorly specified prompts may result in incomplete or inconsistent test cases. Therefore, this study uses the same standardized prompt for all evaluated LLMs, ensuring that the comparison reflects differences in the models rather than variations in the prompting strategy.

3 Related Work

The application of Large Language Models (LLMs) to automated unit test generation has attracted increasing attention in recent years. Existing studies have investigated structural coverage, fault detection capability, maintainability, and the impact of contextual information during test generation. Although these studies demonstrate the potential of LLMs for software testing, they differ in evaluation criteria, experimental settings, and research objectives.

Before the adoption of LLMs, automated unit test generation was primarily dominated by search-based and random-based approaches. [21] compared developer-written test suites with tests generated by EvoSuite and Randoop. Although the automatically generated tests achieved structural coverage comparable to manually written tests, they consistently obtained lower mutation scores, indicating weaker fault detection capability. The authors concluded that coverage alone is insufficient to assess test quality, highlighting the importance of mutation analysis.

With the emergence of LLMs, researchers investigated whether these models could overcome the limitations of traditional automated approaches. [5] compared ChatGPT-generated unit tests with those produced by Pynguin. ChatGPT frequently achieved comparable or higher line coverage while generating more readable tests, but many of the generated tests contained weak assertions, reducing their effectiveness in detecting behavioral faults. These findings indicate that higher structural coverage does not necessarily imply higher test quality.

Subsequent studies investigated factors influencing the effectiveness of LLM-generated tests. [8] evaluated the impact of providing project-wide contextual information to GitHub Copilot and found that richer context significantly improves the correctness and usefulness of generated tests. Rather than comparing LLMs with traditional approaches, this study emphasized the importance of prompt design and contextual information for improving test generation.

While previous studies mainly focused on structural effectiveness, [1] evaluated the internal quality of LLM-generated test suites through the analysis of test smells. Their results showed that GitHub Copilot frequently introduced maintainability problems, such as Assertion Roulette and Magic Numbers, even when the generated tests compiled successfully and achieved satisfactory structural coverage. These findings reinforce that evaluating LLM-generated tests requires quality measures beyond traditional coverage metrics.

More recently, [7] compared ChatGPT-generated unit tests with tests written by undergraduate students. Although both approaches achieved similar code coverage, the human-written tests were considerably more effective at detecting injected faults. This finding reinforces that structural coverage alone is insufficient to characterize test effectiveness and highlights the importance of complementary metrics, particularly mutation analysis.

Collectively, these studies demonstrate significant progress in LLM-based unit test generation while revealing important research

gaps. Previous work has independently investigated structural coverage, mutation analysis, contextual information, and maintainability. However, none jointly compares developer-written test suites and multiple state-of-the-art LLMs using a unified methodology that simultaneously evaluates structural coverage, mutation coverage, test strength, and class-level cyclomatic complexity. This work addresses this gap by providing a comprehensive empirical evaluation across multiple real-world Java projects, offering a broader understanding of the capabilities and limitations of modern LLMs for automated unit test generation.

4 Empirical Study Setup

This section describes the empirical study conducted to compare developer-written unit tests and LLM-generated unit tests. We first present the research questions that guide the study. Then, we describe the selected subject projects, the language models used, the standardized prompt and evaluation tools, and the experimental procedure adopted to collect and compare the results.

4.1 Research Questions

The practice of unit testing still demands effective approaches for producing test suites that not only exercise the source code but also detect defects. Developer-written test suites are widely adopted in real-world software projects and often reflect domain knowledge and developers' testing expertise. More recently, Large Language Models (LLMs) have emerged as a promising alternative for automatically generating unit test suites, with the potential to reduce the effort required from developers.

This study compares developer-written and LLM-generated unit test suites with respect to structural coverage, fault-detection capability, and their effectiveness when testing classes with different levels of code complexity. To guide this empirical investigation, we address the following research questions:

RQ1: How effective are LLM-generated test suites compared to developer-written test suites in terms of code coverage?

RQ2: How effective are LLM-generated test suites compared to developer-written test suites in terms of mutation coverage and test strength?

RQ3: How does code complexity affect the coverage and defect detection capability of developer-written and LLM-generated test suites?

RQ4: How do different LLMs compare to each other when generating unit test suites for the same projects?

4.2 Project Selection

Five open-source projects from the Apache Software Foundation were selected for the empirical evaluation: Apache Commons Lang, Apache Commons Collections, Apache Commons Compress, Apache Commons CLI, and Apache Commons BCEL. These projects were selected according to three criteria: (i) they use the Maven build system [3], facilitating project compilation and test automation; (ii) they include developer-written unit test suites, providing a reliable baseline for comparison with LLM-generated test suites; and (iii) they are compatible with the PIT mutation testing tool [6], allowing mutation analysis without requiring modifications to the projects.

Table 1: Selected projects and their main characteristics

Project	Version	# Classes	LOC
Collections	4.6.0	429	29954
Compress	1.29.0	501	23475
Lang	3.20.1	278	53835
CLI	1.11.0	87	11243
BCEL	6.13.0	672	48150
Total	–	1967	166657

Table 1 summarizes the main characteristics of the selected projects, including the analyzed version, the number of classes in the codebase, and the size in lines of source code (LOC).

Commons Lang provides extensions to Java Core utility classes, such as handlers for strings, numbers, and reflection. Commons Collections provides additional implementations of data structures and collections that are not present in the standard library. Commons Compress defines APIs for handling compression and archiving formats, such as ZIP, TAR, and BZIP2. Commons CLI provides APIs for parsing command-line options passed to Java programs, while Commons BCEL provides functionality for analyzing, creating, and manipulating Java class files and bytecode. These projects were selected because they are representative in terms of size, domain, and variety of implemented logic, and because they provide manually written regression tests, which makes them suitable for the proposed evaluation.

4.3 Language Models Used

Three large language models specialized in source-code generation were employed for the automatic generation of unit test suites: Opus 4.5, GPT-5.1 Codex Max, and Sonnet 4.5. The selection of these models considered the availability of tools specifically trained for programming tasks, in order to maximize the quality of the automatically generated test cases.

Each LLM received exactly the same input prompt requesting the generation of unit tests for the project classes. This prompt standardization ensured equivalent comparison conditions across the models, reducing the influence of prompt variability and focusing the analysis on the ability of each LLM to generate test cases. All selected models support Java and are capable of producing JUnit code, which is the testing framework used in the analyzed projects.

4.4 Prompt and Evaluation Tools

The prompt used for test generation is presented in Appendix A. It provides a standardized set of instructions for generating JUnit 5 unit tests for concrete Java classes with public behavior. The prompt defines the eligibility criteria for production classes, the expected structure of the generated tests, and constraints such as preserving the production code, avoiding unnecessary mocks, and producing meaningful assertions. By using the same prompt for all models and projects, we ensured that the comparison was conducted under equivalent generation conditions.

To evaluate the effectiveness of both developer-written and LLM-generated test suites, we used JaCoCo and PIT. JaCoCo was used to

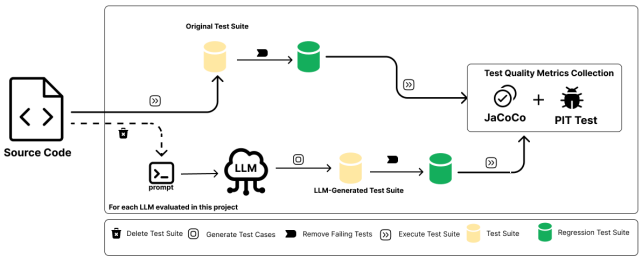

Figure 1: Overview of the experimental procedure adopted in this study.

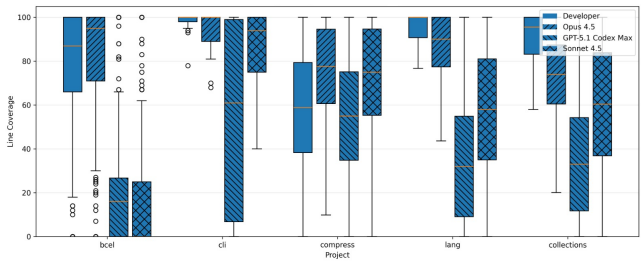

Figure 2: Distribution of JaCoCo line coverage by project and test-suite origin.

collect structural coverage metrics, including line coverage, branch coverage, and cyclomatic complexity. PIT was used to perform mutation testing by introducing artificial changes into the production code and measuring how many of them were detected by the test suites. In this study, mutation coverage and test strength were used as indicators of defect detection capability. Together, these tools allowed us to compare the test suites not only by the amount of code they execute, but also by their ability to reveal faults.

4.5 Experimental Procedure

Figure 1 illustrates the experimental procedure adopted in this study. For each selected project, we first executed the original developer-written test suite. All failing tests were removed until the suite became completely green, since PIT requires a successful baseline test suite before mutation analysis. After that, we collected the baseline metrics using JaCoCo and PIT. The JaCoCo report was generated with the command `mvn jacoco:report`, and the mutation analysis was performed with `mvn test pitest:mutationCoverage`.

After collecting the metrics for the original suite, all existing test cases were removed from the project, leaving only the production code. Then, using the standardized prompt described in Appendix A, each LLM generated a new unit test suite for the target project. The generated tests were compiled and executed, and any failing test was removed. This filtering process was repeated until the generated suite became completely green.

Once each generated suite was validated, JaCoCo and PIT were executed again to collect the same metrics obtained for the original developer-written suite. This enabled a direct comparison between manual and LLM-generated tests under the same evaluation criteria. The entire process was repeated for each of the three LLMs and each of the five projects. Therefore, fifteen complete LLM-based experimental runs were conducted, in addition to the initial execution of the original tests for each project.

All experiments were conducted in Cursor Pro version 2.1.39, using Java 21. Cursor was used because it provides native AI integration and allows the models to access the complete project context during test generation. All procedures were executed on the same machine to reduce environmental variation.

5 Empirical Study: Results and Analysis

This section reports the results of our empirical study. We answer each research question by comparing developer-written test suites with those generated by each LLM, considering code coverage and mutation-based effectiveness.

5.1 RQ1: Line Coverage

To answer RQ1, we compare the line coverage achieved by each test-suite origin. Figure 2 shows the class-level distributions, while Table 2 reports the descriptive statistics.

Developer-written suites achieved the highest mean line coverage in three out of five projects: Commons CLI, Commons Collections, and Commons Lang. However, Opus 4.5 outperformed the developer-written suites in Commons BCEL and Commons Compress, reaching higher mean and median coverage in both projects. This indicates that LLM-generated tests can be competitive with manually written tests in specific contexts.

Among the LLMs, Opus 4.5 was the most consistent model. Its distributions are closer to the developer-written suites, especially in Commons CLI and Commons Compress. In contrast, GPT-5.1 Codex Max and Sonnet 4.5 showed larger variation and lower medians in several projects, particularly in Commons BCEL, Commons Lang, and Commons Collections. This suggests that these models covered some classes effectively but failed to exercise others.

Overall, the results show that LLM-generated suites can reach high structural coverage, but their effectiveness varies substantially across models and projects. Developer-written tests remain more stable, while Opus 4.5 is the closest LLM-generated alternative in terms of line coverage.

RQ1: Developer-written suites achieve the most consistent line coverage overall. However, Opus 4.5 reaches comparable results and outperforms the developer baseline in Commons BCEL and Commons Compress.

5.2 RQ2: Mutation Effectiveness

To answer RQ2, we analyze the relationship between PIT line coverage, mutation coverage, and test strength. Figure 3 shows the class-level relation between PIT line coverage and PIT mutation coverage, while Figure 4 presents the distribution of test strength by project and test-suite origin.

The results show that higher line coverage does not necessarily lead to higher mutation coverage. In Figure 3, several classes appear with high PIT line coverage but substantially lower mutation coverage, especially in Commons BCEL, Commons Compress, and Commons Collections. This indicates that the tests executed the

Table 2: Descriptive statistics of JaCoCo line coverage by project and test-suite origin

Project	Test-Suite Origin	Mean (%)	Median (%)	Std. Dev.
Commons BCEL	Developer	75.27	87.00	**30.58**
Commons BCEL	Opus 4.5	**80.11**	**95.00**	28.05
Commons BCEL	GPT-5.1 Codex Max	19.24	16.00	23.90
Commons BCEL	Sonnet 4.5	17.49	0.00	25.66
Commons CLI	Developer	**98.47**	**100.00**	3.68
Commons CLI	Opus 4.5	94.69	**100.00**	8.23
Commons CLI	GPT-5.1 Codex Max	54.71	61.00	**40.16**
Commons CLI	Sonnet 4.5	86.40	94.00	16.48
Commons Collections	Developer	**88.37**	**95.54**	18.33
Commons Collections	Opus 4.5	74.47	73.99	20.00
Commons Collections	GPT-5.1 Codex Max	36.93	33.00	31.52
Commons Collections	Sonnet 4.5	54.52	60.39	**34.89**
Commons Compress	Developer	59.24	58.85	**30.55**
Commons Compress	Opus 4.5	**73.75**	**77.62**	25.14
Commons Compress	GPT-5.1 Codex Max	57.40	55.00	29.95
Commons Compress	Sonnet 4.5	69.50	75.00	29.18
Commons Lang	Developer	**96.26**	**100.00**	13.79
Commons Lang	Opus 4.5	84.36	90.00	18.62
Commons Lang	GPT-5.1 Codex Max	41.77	32.00	34.02
Commons Lang	Sonnet 4.5	52.04	58.00	**34.18**

code but were not always able to detect the behavioral changes introduced by PIT. Therefore, structural coverage alone is insufficient to characterize test-suite effectiveness.

Developer-written suites present the strongest mutation behavior overall. This is particularly visible in Commons CLI and Commons Collections, where developer tests are concentrated in regions with both high line coverage and high mutation coverage. Figure 4 also shows that developer-written tests usually maintain high test strength, suggesting stronger assertions and more effective fault detection in mature manual suites.

LLM-generated suites show more variable results. Opus 4.5 is the closest model to the developer baseline, with competitive mutation behavior in several projects and particularly strong results in Commons BCEL and Commons Lang. Sonnet 4.5 performs well in Commons Compress, where its mutation effectiveness is close to or above the other origins. GPT-5.1 Codex Max, in contrast, often achieves lower mutation coverage, even when its test strength is high in some projects. This pattern suggests that its tests can kill mutants when they reach relevant behavior, but they exercise a smaller portion of the code.

RQ2: Developer-written suites show stronger mutation effectiveness overall. LLM-generated suites can be competitive in specific projects, especially Opus 4.5 and Sonnet 4.5, but high line coverage alone does not guarantee high defect-detection capability.

5.3 RQ3: Complexity Sensitivity

To answer RQ3, we analyze the effect of cyclomatic complexity on coverage and mutation-based effectiveness. Figure 5 shows the class-level relation between complexity and PIT mutation coverage, while Table 3 summarizes the results for high-complexity classes.

Overall, higher complexity is associated with lower and more variable mutation coverage. This effect is visible in Figure 5, especially for Commons BCEL, Commons Lang, and Commons Collections, where high-complexity classes frequently appear with low mutation coverage. The effect is stronger for GPT-5.1 Codex Max and Sonnet 4.5, indicating that these models struggle more to exercise complex behavior.

Table 3 confirms that no origin dominates all projects. Developer-written tests are strongest in Commons CLI, while Opus 4.5 performs best in Commons BCEL and Commons Lang. Sonnet 4.5 achieves the best results in Commons Compress. In Commons Collections, developer-written tests achieve the highest line coverage, but Opus 4.5 obtains the highest mutation coverage.

The gap between line coverage and mutation coverage remains relevant in several projects, particularly in Commons Collections and Commons Compress. This shows that covering complex code is not sufficient: tests must also include effective assertions to detect behavioral changes.

RQ3: Code complexity reduces test effectiveness, mainly for LLM-generated suites. Developer-written tests are more robust overall, but Opus 4.5 and Sonnet 4.5 outperform them in specific high-complexity scenarios.

5.4 RQ4: LLM Comparison

To answer RQ4, we compare the three LLMs using line coverage, mutation coverage, and test strength. Table 4 summarizes the ranking by project and metric.

Opus 4.5 is the most consistent model. It achieves the highest line coverage in all five projects and the highest mutation coverage in four of them. It also obtains the best test strength in Commons Collections and Commons Lang. These results indicate that Opus 4.5

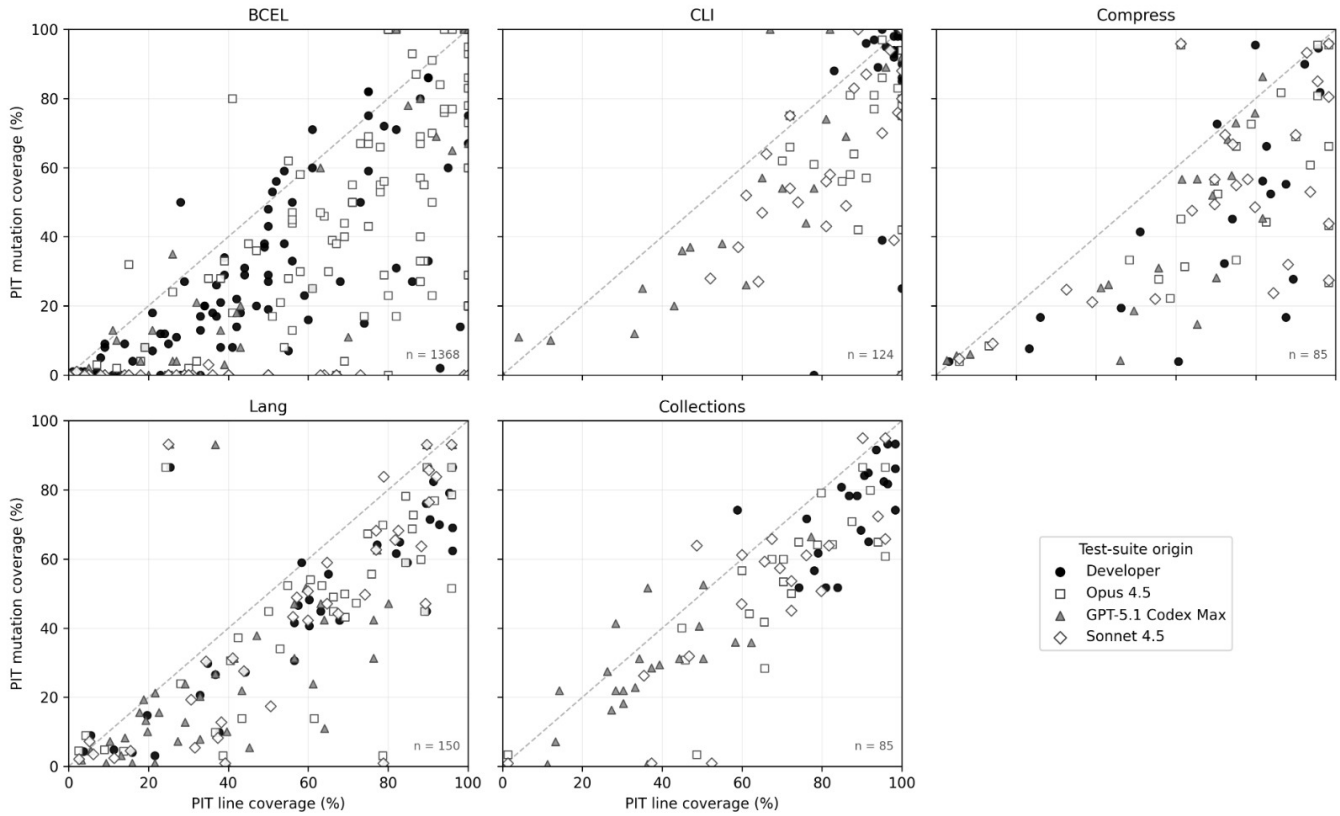

Figure 3: Relationship between PIT line coverage and PIT mutation coverage at class level, grouped by project and test-suite origin. The dashed diagonal line represents equal line and mutation coverage.

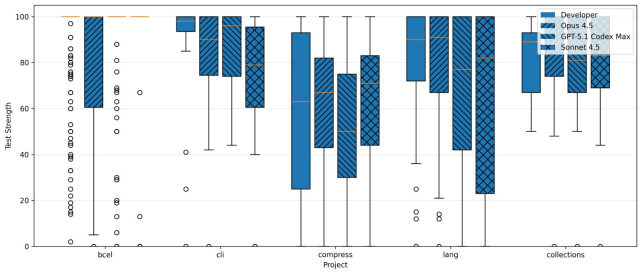

Figure 4: Distribution of PIT test strength by project and test-suite origin.

provides the best balance between structural coverage and defect-detection capability.

Sonnet 4.5 ranks second overall. Its strongest result occurs in Commons Compress, where it achieves the highest mutation coverage and test strength among the LLMs. However, its performance is less stable across the remaining projects, especially in BCEL and Lang.

GPT-5.1 Codex Max presents the weakest overall performance. Although it reaches the highest test strength in Commons BCEL and Commons CLI, its line coverage and mutation coverage are consistently lower. This suggests that its tests can be effective when they reach relevant code, but they do not exercise enough of the system.

RQ4: Opus 4.5 is the best overall LLM, followed by Sonnet 4.5. GPT-5.1 Codex Max shows isolated gains in test strength, but lower overall coverage and mutation effectiveness.

5.5 Discussion

Our study shows that LLM-generated test suites can be competitive with developer-written suites, but their effectiveness depends on the evaluation metric, the project characteristics, and the model. In terms of line coverage, LLMs can exercise a substantial portion of the code and, in some projects, even surpass the developer baseline. This was observed mainly for Opus 4.5 in Commons BCEL and Commons Compress, as shown in Table 2 and Figure 2. However, this behavior was not uniform, since developer-written suites remained stronger in Commons CLI, Commons Collections, and Commons Lang. This variation suggests that the effectiveness of LLM-generated tests is influenced by characteristics of the target project, indicating that current models perform better in some contexts than others rather than consistently outperforming manually developed test suites.

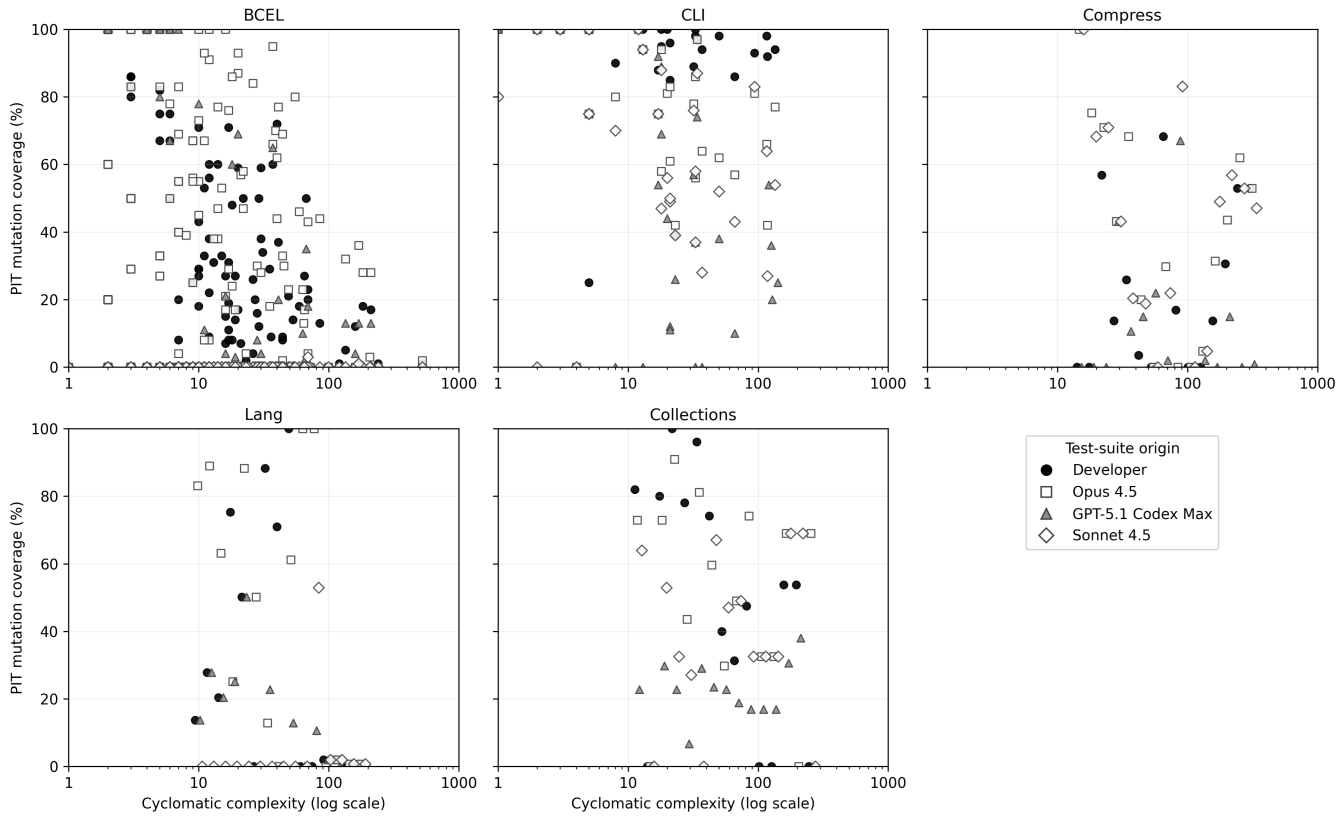

Figure 5: Cyclomatic complexity versus PIT mutation coverage at class level, grouped by project and test-suite origin. The x-axis is shown in log scale.

The comparison between PIT line coverage and mutation coverage provides a more stringent view of test effectiveness. As shown in Figure 3, several classes appear below the diagonal, meaning that they were executed by the tests but many of their mutants survived. This result reinforces that line coverage alone is not sufficient to assess test quality, since executing code does not imply checking its behavior with effective oracles [11]. Our results suggest that current LLMs are generally successful at producing executable tests that exercise common execution paths, but they are less effective at generating assertions capable of detecting subtle behavioral changes. Therefore, mutation-based metrics remain essential to reveal weaknesses that structural coverage alone may hide.

Our mutation results indicate that developer-written suites are still more reliable for defect detection. This is particularly visible in Commons CLI and Commons Collections, where developer tests are concentrated in regions with both high PIT line coverage and high mutation coverage. Nevertheless, the study also shows that LLM-generated tests can complement manual suites. Opus 4.5 and Sonnet 4.5 outperformed the developer baseline in specific cases, especially in Commons Compress and, for some metrics, Commons BCEL and Commons Lang. This suggests that LLMs may help expand test suites by exercising additional scenarios, although their assertions may still be weaker or less complete in some cases. One possible explanation is that LLMs infer assertions primarily from recurring coding patterns, whereas developers can leverage domain knowledge to validate more subtle behavioral properties.

The relationship between mutation coverage and test strength also deserves attention. Figure 4 shows that some LLM-generated suites reach high test strength, even when their overall mutation coverage is low. This indicates that the generated tests can be effective over the code they reach, but they may exercise only a limited portion of the program. Similar concerns have been reported in the literature, where test oracles and behavioral checks are identified as central factors for distinguishing code execution from actual fault detection [11, 13]. Therefore, improving only the quality of assertions may not be sufficient if generated test suites still fail to explore relevant execution paths.

The analysis of cyclomatic complexity reinforces this interpretation. As shown in Figure 5 and Table 3, high-complexity classes tend to reduce both coverage and mutation effectiveness, especially for LLM-generated suites. This result indicates that current LLMs still struggle to generate tests that exercise complex control-flow paths and validate their behavior effectively. Highly complex classes typically contain a larger number of execution paths and corner cases, making it more difficult to infer representative test scenarios directly from source code. In these cases, developer-written tests remain more robust, although Opus 4.5 and Sonnet 4.5 achieved competitive results in some high-complexity scenarios.

Table 3: Effectiveness of test suites on high-complexity classes

Project	Origin	High-Cxty Line Cov. (%)	High-Cxty Mutation Cov. (%)	High-Cxty Test Strength (%)	Gap
Commons BCEL	Developer	12.33	5.75	63.42	**6.58**
Commons BCEL	Opus 4.5	**18.92**	**14.42**	74.92	4.50
Commons BCEL	GPT-5.1 Codex Max	8.00	3.58	**81.58**	4.42
Commons BCEL	Sonnet 4.5	0.83	0.08	63.92	0.75
Commons CLI	Developer	**97.29**	**89.93**	**91.00**	7.36
Commons CLI	Opus 4.5	88.71	68.00	76.29	20.71
Commons CLI	GPT-5.1 Codex Max	39.36	28.57	78.00	10.79
Commons CLI	Sonnet 4.5	76.64	53.36	69.21	**23.29**
Commons Collections	Developer	**75.92**	49.10	57.31	**26.82**
Commons Collections	Opus 4.5	65.67	**51.82**	71.45	13.86
Commons Collections	GPT-5.1 Codex Max	31.16	19.69	**72.21**	11.48
Commons Collections	Sonnet 4.5	58.20	38.35	67.40	19.84
Commons Compress	Developer	48.86	22.35	40.37	**26.51**
Commons Compress	Opus 4.5	56.68	40.13	51.08	16.55
Commons Compress	GPT-5.1 Codex Max	27.48	11.82	30.22	15.66
Commons Compress	Sonnet 4.5	**60.42**	**42.48**	**58.17**	17.93
Commons Lang	Developer	38.72	30.14	56.84	8.58
Commons Lang	Opus 4.5	**57.25**	**45.10**	**65.91**	12.16
Commons Lang	GPT-5.1 Codex Max	29.86	12.37	48.97	**17.49**
Commons Lang	Sonnet 4.5	12.18	3.90	20.99	8.29

Table 4: Ranking of LLMs by project and evaluation metric

Project	LLM	Line Coverage (%)	Mutation Coverage (%)	Test Strength (%)
Commons BCEL	Opus 4.5	**80.11**	**30.56**	79.74
	GPT-5.1 Codex Max	19.24	3.80	**96.23**
	Sonnet 4.5	17.49	0.01	92.05
Commons CLI	Opus 4.5	**94.69**	**77.71**	82.94
	GPT-5.1 Codex Max	54.71	49.13	**86.87**
	Sonnet 4.5	86.40	64.74	74.77
Commons Collections	Opus 4.5	**74.47**	**60.25**	**79.67**
	GPT-5.1 Codex Max	36.93	29.33	76.29
	Sonnet 4.5	54.52	52.58	78.08
Commons Compress	Opus 4.5	**73.75**	49.00	59.24
	GPT-5.1 Codex Max	57.40	28.12	47.32
	Sonnet 4.5	69.50	**49.08**	**63.72**
Commons Lang	Opus 4.5	**84.36**	**58.32**	**77.67**
	GPT-5.1 Codex Max	41.77	26.22	66.08
	Sonnet 4.5	52.04	36.93	61.90

The comparison among LLMs shows that their performance is not homogeneous. Table 4 indicates that Opus 4.5 was the most consistent model, achieving the best balance between line coverage, mutation coverage, and test strength. Sonnet 4.5 showed strong results in Commons Compress, but its performance varied more across projects. GPT-5.1 Codex Max achieved high test strength in isolated cases, but its lower coverage and mutation results indicate limited overall effectiveness. Although our study does not investigate the internal reasons for these differences, the results suggest that current LLMs differ not only in their ability to generate executable tests but also in how effectively they explore program behavior and construct meaningful assertions.

Overall, our findings suggest that LLM-generated tests should be interpreted as a complementary testing strategy rather than a direct replacement for developer-written suites. They can increase structural coverage and reveal additional testing opportunities, but developer-written tests remain more effective for consistently detecting faults, especially in complex code. More broadly, our results indicate that future advances in LLM-based test generation should focus not only on increasing coverage, but also on improving the quality of test oracles and the exploration of complex program

behaviors, allowing AI-generated tests to become a more effective complement to developer expertise.

6 Threats to Validity and Limitations

There are several threats to the validity of this study. First, the generated test suites depend on the prompt used to instruct the LLMs. Although we used the same standardized prompt for all models to ensure a fair comparison, Prior work has shown that LLM performance can vary depending on prompt format, example selection, example ordering, and small wording changes [16, 26]. Therefore, different prompting strategies could lead to different generated test suites and, consequently, different coverage and mutation results.

Another threat concerns the stochastic behavior of LLMs. Even when using the same prompt and project context, LLMs may generate different outputs across executions [17]. In this study, we controlled the experimental environment and used the same procedure for all models, but we did not run multiple independent generations for each model-project pair. Thus, the reported results should be interpreted as evidence for the generated suites analyzed in this experiment, rather than as the complete distribution of possible outputs for each model.

The filtering of failing tests may also affect the results. Since PIT requires a passing test suite before mutation analysis, tests that failed to compile or failed during execution were removed. This step was necessary to evaluate all suites under the same regression-testing setting. However, removing tests may also discard scenarios that exercised relevant behavior, which can influence the final coverage and mutation scores.

The results are also limited by the selected subject systems. We evaluated five Apache Commons projects written in Java and built with Maven. These projects are real, mature, and widely used open-source systems, but they may not represent other domains, programming languages, testing frameworks, or industrial architectures. Systems with graphical interfaces, external services, concurrency, or strong integration dependencies may impose different challenges for LLM-generated tests.

The choice of models is another limitation. We evaluated Opus 4.5, GPT-5.1 Codex Max, and Sonnet 4.5. Other models, model versions, IDE integrations, decoding settings, or context-management strategies may produce different results. Since LLM-based tools evolve rapidly, our findings should be interpreted within the specific model versions and experimental conditions used in this study.

There are also limitations related to the metrics used to measure test quality. Line coverage indicates how much code is executed, but it does not guarantee that the executed behavior is properly checked. A test may cover many lines while using weak assertions or failing to validate relevant behavior. For this reason, we also used mutation coverage and test strength. However, mutation testing is itself a proxy for defect-detection capability. Its results depend on the mutation operators implemented by the tool, and some surviving mutants may be equivalent to the original program or may not correspond to realistic faults.

Finally, test strength must be interpreted together with mutation coverage. A suite may achieve high test strength on the mutants it reaches, while still presenting low mutation coverage because it exercises only a small portion of the program. This situation appeared in some LLM-generated suites and reinforces that no single metric is sufficient to characterize test-suite quality. Overall, we mitigated these threats by using the same projects, prompt, tools, environment, and evaluation procedure for all test-suite origins, but the results should be interpreted within these constraints.

7 Conclusion and Future Work

This study investigated the effectiveness of unit test suites generated by state-of-the-art Large Language Models in comparison with developer-written test suites. Our results show that LLM-generated tests can achieve competitive structural coverage and, in some projects, even surpass the developer baseline. However, higher line coverage does not necessarily imply better fault-detection capability. When mutation-based metrics are considered, developer-written test suites remain consistently more effective, reinforcing the importance of evaluating both structural and behavioral aspects of test quality.

Our findings also show that the effectiveness of LLM-generated tests depends on the target project, the evaluation metric, and the model. Among the evaluated models, Opus 4.5 achieved the most consistent balance between line coverage, mutation coverage, and test strength, while Sonnet 4.5 obtained competitive results in specific projects and GPT-5.1 Codex Max performed well only in isolated cases. Moreover, high cyclomatic complexity remains a significant challenge for all evaluated LLMs, indicating that generating tests capable of exercising and validating complex program behavior is still an open problem. Overall, these findings suggest that LLM-generated tests are best used as a complement to developer-written suites, combining the productivity benefits of AI with the stronger behavioral reasoning and domain knowledge provided by developers.

As future work, we plan to extend the evaluation to additional projects, programming languages, testing frameworks, and LLMs. We also intend to investigate alternative prompting strategies, perform multiple independent generations to assess the impact of model non-determinism, and explore hybrid approaches that combine LLM-based test generation with traditional automated testing techniques and mutation-guided feedback to improve both structural coverage and fault-detection capability.

References

[1] Victor Anthony Alves, Cristiano Santos, Carla Bezerra, and Ivan Machado. 2024. Detecting test smells in python test code generated by LLM: An empirical study with GitHub copilot. In *Simpósio Brasileiro de Engenharia de Software (SBES)*. SBC, 581–587.

[2] AMAZON WEB SERVICES. 2025. *What is unit testing?* https://aws.amazon.com/pt/what-is/unit-testing/ Definição de testes de unidade fornecida pela AWS — processo de testar a menor unidade funcional de código para garantir a qualidade do software.

[3] Apache Maven. 2026. Apache Maven Project. https://maven.apache.org/. Accessed: 2025-12-08.

[4] Shreya Bhatia, Tarushi Gandhi, Dhruv Kumar, and Pankaj Jalote. 2024. Unit Test Generation using Generative AI : A Comparative Performance Analysis of Autogeneration Tools. arXiv:2312.10622 [cs.SE] https://arxiv.org/abs/2312.10622

[5] Shreya Bhatia, Tarushi Gandhi, Dhruv Kumar, and Pankaj Jalote. 2024. Unit test generation using generative AI: A comparative performance analysis of autogeneration tools. In *Proceedings of the 1st International Workshop on Large Language Models for Code*. 54–61.

[6] Henry Coles, Thomas Laurent, Christopher Henard, Mike Papadakis, and Anthony Ventresque. 2016. PIT: a practical mutation testing tool for Java (demo). In *Proceedings of the 25th International Symposium on Software Testing and Analysis*

(Saarbrücken, Germany) *(ISSTA 2016)*. Association for Computing Machinery, New York, NY, USA, 449–452. doi:10.1145/2931037.2948707

[7] Ronald Díaz-Arrieta, Byron Díaz-Monroy, and Luis Castillo. 2024. Comparative Analysis of the Efficiency of Generation of Unit Test Cases: Manual Methods Versus Automation with LLM. https://ssrn.com/abstract=5185412. Available at SSRN: https://ssrn.com/abstract=5185412 or http://dx.doi.org/10.2139/ssrn.5185412.

[8] Khalid El Haji, Carolin Brandt, and Andy Zaidman. 2024. Using github copilot for test generation in python: An empirical study. In *Proceedings of the 5th ACM/IEEE International Conference on Automation of Software Test (AST 2024)*. 45–55.

[9] Danielle Gonzalez, Joanna C.S. Santos, Andrew Popovich, Mehdi Mirakhorli, and Mei Nagappan. 2017. A Large-Scale Study on the Usage of Testing Patterns That Address Maintainability Attributes: Patterns for Ease of Modification, Diagnoses, and Comprehension. In *2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR)*. IEEE, 391–401. doi:10.1109/msr.2017.8

[10] Emil Hansson and Oskar Ellréus. 2023. *Code Correctness and Quality in the Era of AI Code Generation: Examining ChatGPT and GitHub Copilot.* Ph. D. Dissertation. Linnaeus University. https://urn.kb.se/resolve?urn=urn:nbn:se:lnu:diva-121545

[11] Soneya Binta Hossain and Matthew B. Dwyer. 2022. A Brief Survey on Oracle-based Test Adequacy Metrics. *arXiv preprint arXiv:2212.06118* (2022).

[12] Shaokang Jiang and Daye Nam. 2025. An Empirical Study of Developer-Provided Context for AI Coding Assistants in Open-Source Projects. arXiv:2512.18925 [cs.SE] https://arxiv.org/abs/2512.18925

[13] Michael Konstantinou, Renzo Degiovanni, and Mike Papadakis. 2024. Do LLMs generate test oracles that capture the actual or the expected program behaviour? *arXiv preprint arXiv:2410.21136* (2024).

[14] Divya Kumar and K.K. Mishra. 2016. The Impacts of Test Automation on Software's Cost, Quality and Time to Market. *Procedia Computer Science* 79 (2016), 8–15. doi:10.1016/j.procs.2016.03.003 Proceedings of International Conference on Communication, Computing and Virtualization (ICCCV) 2016.

[15] Chun Li. 2022. Mobile GUI test script generation from natural language descriptions using pre-trained model. 112–113. doi:10.1145/3524613.3527809

[16] Yao Lu, Max Bartolo, Alastair Moore, Sebastian Riedel, and Pontus Stenetorp. 2021. Fantastically Ordered Prompts and Where to Find Them: Overcoming Few-Shot Prompt Order Sensitivity. *arXiv preprint arXiv:2104.08786* (2021).

[17] Shuyin Ouyang, Jie M. Zhang, Mark Harman, and Meng Wang. 2023. An Empirical Study of the Non-determinism of ChatGPT in Code Generation. *arXiv preprint arXiv:2308.02828* (2023).

[18] Martín Rodríguez, Gustavo Rossi, and Alejandro Fernandez. 2025. Evaluating Large Language Models for the Generation of Unit Tests with Equivalence Partitions and Boundary Values. arXiv:2505.09830 [cs.SE] https://arxiv.org/abs/2505.09830

[19] Per Runeson. 2006. A Survey of Unit Testing Practices. *IEEE Software* 23 (07 2006). doi:10.1109/MS.2006.91

[20] Domenico Serra, Giovanni Grano, Fabio Palomba, Filomena Ferrucci, Harald C. Gall, and Alberto Bacchelli. 2019. On the Effectiveness of Manual and Automatic Unit Test Generation: Ten Years Later. In *2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR)*. 121–125. doi:10.1109/MSR.2019.00028

[21] Domenico Serra, Giovanni Grano, Fabio Palomba, Filomena Ferrucci, Harald C Gall, and Alberto Bacchelli. 2019. On the effectiveness of manual and automatic unit test generation: ten years later. In *2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR)*. IEEE, 121–125.

[22] Saeed Shamshiri, José M. Rojas, Juan P. Galeotti, Neil Walkinshaw, and Gordon Fraser. 2018. How Do Automatically Generated Unit Tests Influence Software Maintenance?. In *2018 IEEE 11th International Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 250–261. https://ieeexplore.ieee.org/document/8367053

[23] Chen Yang, Junjie Chen, Bin Lin, Ziqi Wang, and Jianyi Zhou. 2025. Advancing Code Coverage: Incorporating Program Analysis with Large Language Models. arXiv:2404.04966 [cs.SE] https://arxiv.org/abs/2404.04966

[24] Burak Yetistiren, Isik Ozsoy, and Eray Tuzun. 2022. Assessing the quality of GitHub copilot's code generation. In *Proceedings of the 18th International Conference on Predictive Models and Data Analytics in Software Engineering* (Singapore, Singapore) *(PROMISE 2022)*. Association for Computing Machinery, New York, NY, USA, 62–71. doi:10.1145/3558489.3559072

[25] Shengcheng Yu, Chunrong Fang, Yuchen Ling, Chentian Wu, and Zhenyu Chen. 2023. LLM for Test Script Generation and Migration: Challenges, Capabilities, and Opportunities. arXiv:2309.13574 [cs.SE] https://arxiv.org/abs/2309.13574

[26] Tony Z. Zhao, Eric Wallace, Shi Feng, Dan Klein, and Sameer Singh. 2021. Calibrate Before Use: Improving Few-Shot Performance of Language Models. *arXiv preprint arXiv:2102.09690* (2021).

A Prompt Used for Unit Test Generation

```
You are a senior Java developer writing unit
tests for a real-world codebase.

Given the source code of a Java project,
identify all production classes that meet
the following criteria:
- Located under src/main/java
- Are concrete classes (not abstract)
- Are not interfaces
- Are not enums
- Declare at least one public method

For each eligible class, generate JUnit 5
unit tests following these rules:
- Tests must compile successfully
- Use realistic and idiomatic test structures
- Include meaningful assertions reflecting
real developer intent
- Cover common use cases and a small number
of relevant edge cases
- Use clear and natural test method and
variable names
- Avoid exhaustive or artificial edge cases
- Avoid mocks unless strictly necessary
- Do not include any comments in the test code

Important constraints:
- Generate one test class per production
class
- Place tests under src/test/java, mirroring
the package structure
- Do not generate tests for classes that do
not meet the eligibility criteria
- Do not modify the production code
```